

AutoRestTest

An AI-Powered Collaborative Platform for Automated REST API Testing

SE 801

Software Project Lab 3 Technical Report

Submitted by:

Mohammad Ismail Hossain

Roll: **BSSE 1433**

Institute of Information Technology

University of Dhaka

Supervised by:

Toukir Ahammed

Lecturer

Institute of Information Technology

University of Dhaka

Submitted to:

SPL-3 Program Committee

Submission Date: 02-08-26

Table of Contents

1. Project overview	4
1.1 Project Title	4
1.2 Problem Statement	4
1.3 Objectives	4
1.4 Scope	4
1.5 Deliverables	5
2. Requirements Analysis	5
2.1 Quality Function Deployment	5
2.1.1 Normal Requirements	5
Functional Requirements	5
Non-Functional Requirements	6
2.1.2 Expected Requirements	6
Functional Requirements	6
Non-Functional Requirements	6
2.1.3 Exciting Requirements	6
Functional Requirements	7
Non-Functional Requirements	7
2.2 Stakeholders	7
2.3 Usage Scenario	7
2.3.1 Authentication System	7
2.3.2 Project Management	8
2.3.3 API Specification Management	8
2.3.4 Test Suite Management	8
2.3.5 Test Execution	9
2.3.6 Test Results and Reports	9
2.3.7 Team Collaboration	10
3. System Modeling	11
3.1 Use Case Diagrams	11
3.1.1 Level 0 — AutoRestTest Overview	11
3.1.2 Level 1 — Full System	12
3.1.3 Level 1.1 — Authentication system	13
3.1.4 Level 1.2 — Project Management	14
3.1.5 Level 1.3 — API Specification Management	15
3.1.6 Level 1.4 — Test Suite Management	16
3.1.7 Level 1.5 — Test Execution	17
3.1.8 Level 1.6 — Test Results & Reports	18
3.1.9 Level 1.7 — Team Collaboration	19

3.2 Activity Diagrams	20
3.2.1 Authentication system	20
3.2.2 Project Management	21
3.2.3 API Specification Management	22
3.2.4 Test Suite Management	23
3.2.5 Test Execution	24
3.2.6 Test Results & Reports	25
3.2.7 Team Collaboration	26
4. Data Modeling	27
4.1 ER Diagram	27
4.2 Database schema	28
5. AI Engineering Design	30
5.1 AI Pipeline	30
5.2 Model Selection	32
5.3 Prompt Design	33
6. Preliminary Test Plan	34
6.1 High-Level Description of Testing Goals	34
6.2 Test Cases	34
7. Timeline	36
8. Conclusion	37
9. References	37

List of Figures

Figure 1: Use case diagram — Level 0: AutoRestTest system overview	11
Figure 2: Use case diagram — Level 1: Full system	12
Figure 3: Use case diagram — Level 1.1: Authentication	13
Figure 4: Use case diagram — Level 1.2: Project & Endpoint Management	14
Figure 5: Use case diagram — Level 1.3: API Specification Management	15
Figure 6: Use case diagram — Level 1.4: Test Suite Management	16
Figure 7: Use case diagram — Level 1.5: Test Execution	17
Figure 8: Use case diagram — Level 1.6: Test Results & Reports	18
Figure 9: Use case diagram — Level 1.7: Team Collaboration	19
Figure 10: Activity diagram — Authentication	20
Figure 11: Activity diagram — Project & Endpoint Management	21
Figure 12: Activity diagram — API Specification Management	22
Figure 13: Activity diagram — Test Suite Management	23
Figure 14: Activity diagram — Test Execution	24
Figure 15: Activity diagram — Test Results & Reports	25
Figure 16: Activity diagram — Team Collaboration	26
Figure 17: Entity Relationship Diagram	27
Figure 18: AI system workflow diagram	32
Figure 18: Timeline (Gantt Chart)	36

1. Project overview

1.1 Project Title

AutoRestTest: An AI-Powered Platform for Automated REST API Testing

1.2 Problem Statement

As modern web services increasingly rely on REST APIs, ensuring their correctness is critical. As APIs grow in size and complexity, testing them becomes progressively harder because of large input spaces, interdependent operations, and the need for valid request sequences. Traditional automated testing tools often fail to explore these dependencies effectively, leading to low coverage, missed edge cases, and weak fault detection, while manual testing is time-consuming, repetitive, error-prone, and unable to scale with continuous development. Recent research such as AutoRestTest [1] shows that dependency-aware, reinforcement-learning and LLM-powered test generation can substantially improve coverage and fault detection for REST APIs. At the same time, development teams need a centralized, collaborative environment in which specifications, test suites, execution results, and reports can be stored, shared, and reused over time. Existing solutions rarely combine an intelligent testing engine with such a platform, and this gap is the problem this project addresses.

1.3 Objectives

The main objectives of this project are as follows:

- To design and develop an AI-powered, collaborative SaaS platform for automated REST API testing, accessible through a centralized web dashboard.
- To integrate an intelligent testing engine that uses dependency-aware, reinforcement-learning and LLM-based request generation, inspired by the AutoRestTest [1] approach, to maximize coverage and fault detection.
- To allow users to upload OpenAPI specifications, or generate them with AI-assisted analysis, and to extract their endpoints automatically.
- To automatically generate, save, and reuse test suites for each project.
- To provide one-click, asynchronous test execution with real-time status updates and persistent execution history.
- To present clear results and analytics, including status-code distribution, coverage, and server failures, together with LLM-based failure explanations and exportable PDF/CSV reports.
- To enable secure team collaboration through project sharing and role-based access control.

1.4 Scope

The scope of this project defines the boundaries of the platform and the extent of the functionality it delivers. AutoRestTest is a web-based SaaS application for the functional

testing of RESTful web services described by OpenAPI 3.0 specifications. Within this boundary it supports the complete testing workflow through a single centralized dashboard, from user and project management to specification handling, AI-driven test generation and execution, result analysis and reporting, and multi-user collaboration, so that development teams can automate REST API testing and reduce manual effort.

Certain areas are intentionally outside the scope of this project. The platform targets REST APIs only and does not address SOAP, GraphQL, or gRPC services, nor specification formats other than OpenAPI 3.0. It focuses on functional and fault-detection testing rather than load, stress, or performance testing, and it does not modify, deploy, or manage the systems under test. The platform further assumes that the target API is reachable over the network and that any credentials required to access protected endpoints are provided by the user.

1.5 Deliverables

The main deliverables of this project are as follows:

- A deployed web application (frontend and backend) that provides the AutoRestTest platform through a centralized, browser-based dashboard.
- An AI-powered testing engine, deployed as an independent service, that performs dependency-aware, reinforcement-learning and LLM-based test generation and execution.
- A PostgreSQL database implementing the platform's data model for users, projects, specifications, test suites, executions, results, and collaboration.
- A complete, documented source-code repository covering the frontend, backend, and testing engine.
- This technical report, documenting the project's overview, requirements, system and data models, AI engineering design, preliminary test plan and timeline.

2. Requirements Analysis

2.1 Quality Function Deployment

2.1.1 Normal Requirements

Explicitly requested core features; user satisfaction is proportional to how well they are met.

Functional Requirements

- ☐ **FR-N1 — User Authentication:** register, log in, and maintain secure sessions.
- ☐ **FR-N2 — Project Management:** create, open, rename, and delete API testing projects from a central dashboard.
- ☐ **FR-N3 — Specification Upload:** upload an OpenAPI 3.0 file; the system parses it and lists the extracted endpoints.

- ☐ **FR-N4 — Test Execution:** run automated requests against the target API from a specification and collect the responses.
- ☐ **FR-N5 — Result Viewing:** view each run's outcome per endpoint (method, request, response, status code, pass/fail).
- ☐ **FR-N6 — Completion Notifications:** notify the user when a run completes, fails, or errors.

Non-Functional Requirements

- ☐ **NFR-N1 — Usability:** the whole workflow is driven from one intuitive web dashboard, usable without special training.
- ☐ **NFR-N2 — Responsiveness:** routine interactions (dashboard, navigation) load within a couple of seconds.

2.1.2 Expected Requirements

Implicitly assumed; their absence causes dissatisfaction even though users rarely ask for them by name.

Functional Requirements

- ☐ **FR-E1 — Automatic Test Suite Generation:** automatically generate a comprehensive, dependency-aware suite of test cases from the specification.
- ☐ **FR-E2 — Save & Reuse Suites:** persist named test suites for re-execution without reconfiguration.
- ☐ **FR-E3 — Execution History:** keep every past run with its timestamp and outcome for later review.
- ☐ **FR-E4 — Reports & Analytics:** per-run summary of HTTP status-code distribution, coverage, and server failures.
- ☐ **FR-E5 — Export Reports:** download a run's results as PDF or CSV.
- ☐ **FR-E6 — Team Collaboration:** invite members to a project and assign roles (admin, tester, viewer).

Non-Functional Requirements

- ☐ **NFR-E1 — Security & Data Privacy:** passwords hashed, routes JWT-protected, and each user's and project's data isolated via role-based access control.
- ☐ **NFR-E2 — Reliability:** long-running executions run asynchronously without blocking the UI and report status accurately.
- ☐ **NFR-E3 — Data Persistence:** specifications, suites, runs, and results are stored durably and remain available across sessions.

2.1.3 Exciting Requirements

Delighters beyond expectation; their absence isn't penalised, but their presence differentiates the platform.

Functional Requirements

- ☐ **FR-X1 — One-Click End-to-End Testing:** a single action runs the entire pipeline — read spec, generate tests, execute, and produce a report.
- ☐ **FR-X2 — AI Failure Explanation:** LLM-generated, plain-language explanation of what caused a failure and which request triggered it.
- ☐ **FR-X3 — AI Specification Generation:** automatically generate an OpenAPI specification from uploaded source code.

Non-Functional Requirements

- ☐ **NFR-X1 — Extensibility:** the AI testing engine is deployed as an independent service, so its generation strategy can evolve without changing the web platform.

2.2 Stakeholders

The key stakeholders of the platform and their interests are summarised below:

- **API Developer:** Uploads specifications and runs tests to catch defects in their REST APIs early.
- **QA Engineer / Tester:** Generates and executes test suites and reviews results and failure explanations.
- **Team Lead / Project Manager:** Creates projects, invites members, assigns roles, and tracks testing progress.
- **Viewer (Client / Stakeholder):** Reviews reports and analytics without modifying tests.

2.3 Usage Scenario

2.3.1 Authentication System

The system will have an authentication system with two options: sign up for new users and log in for users who already have an account.

User Sign Up

A new user will begin registration by visiting the platform's sign-up page and clicking the Create Account button. The user will need to provide a valid email address, a username, and a password. The system will validate the uniqueness of the username and email and the format of the email address.

User Login

Already registered users can log in by providing their email address and password on the login page. The system will maintain user sessions securely, allowing users to remain logged in without having to authenticate repeatedly. Users will also have the option to keep their login details saved for quicker access in future sessions.

2.3.2 Project Management

Once logged in, users will be taken to the main dashboard where all their projects are displayed. The system will allow users to create, manage, and organise multiple API testing projects from a single centralised interface.

Create Project

A user can create a new project by clicking the New Project button on the dashboard. The user will provide a project name and an optional description. Once created, the project will appear on the dashboard and the user will be taken to the project workspace where they can begin setting up their testing environment.

Manage Projects

The dashboard will display all projects belonging to the user along with their names, creation dates, and last activity. Users can open, rename, or delete any project from the dashboard. Users can add multiple API endpoints inside a project. Deleting a project will remove all associated specifications, test suites, and results permanently, following a confirmation prompt from the system.

2.3.3 API Specification Management

Before testing can begin, users need to define the API they want to test. Each project may have an API specification. The system will provide two ways to do this: uploading an existing specification file or generating one automatically from source code.

Upload OpenAPI Specification

Users can upload an OpenAPI specification file in .yaml format directly into their project. The system will parse and validate the uploaded file and display the list of detected API endpoints and operations, making them available for test generation. If the file contains errors or is not a valid specification, the system will notify the user with a description of the problem.

Automated API Specification Generation

For users who do not have an existing specification file, the system will provide an option to generate one automatically. The user can upload their API source code files and the system will analyse them to produce a valid OpenAPI specification. The generated specification will be displayed for review before the user proceeds, and it can be used as the basis for testing in the same way as a manually uploaded file.

2.3.4 Test Suite Management

Once an API specification is available within the project, the system will allow users to generate, save, and reuse test suites for repeated testing.

Automatic Test Suite Generation

After a specification is uploaded or generated, the system will automatically produce a comprehensive set of test cases covering the detected API endpoints and operations. The generated test suite will be displayed to the user showing each test case, its target

endpoint, and the HTTP method that will be used. Users are not required to write or configure any test cases manually.

Save and Reuse Test Suites

The test suite will be saved for future use. Saved test suites will appear in the project's Test Suites section and can be re-executed at any time without regenerating or reconfiguring them. This is particularly useful for regression testing, where the same suite needs to be run repeatedly after API changes.

2.3.5 Test Execution

When a test suite is executed, the system will send automated requests to the target API, collect the responses, and track the progress in real time.

One-Click API Testing

The system will provide a single Run button that, when clicked, triggers the entire testing pipeline automatically. The system will read the active specification, generate test cases, execute all tests against the target API, and produce a full report — all without requiring any additional input from the user. This allows a team to run a complete API test in a single action.

Running Tests

Users can initiate a test run by selecting a test suite and clicking the Run button. The system will display a live progress indicator showing which endpoints are currently being tested and how many have been completed. Users can monitor the execution as it progresses without needing to refresh the page.

Test Alerts and Notifications

Once a test run is complete, the system will send the user a notification confirming completion along with a brief summary of the outcome. If the run encounters a failure — such as the target API being unreachable, an unexpected server error, or a system-level issue during execution — the user will be notified immediately with a description of what went wrong.

2.3.6 Test Results and Reports

After each test run, the system will store the results and provide detailed reporting and analysis options for the user to review.

Viewing Test Results

Users can open any completed test run to see a full breakdown of the results. The results page will show each endpoint that was tested, the HTTP method used, the response received, and whether the test passed or failed. Users can filter the results by endpoint, response status, or HTTP response code to focus on specific areas of interest. In addition, the system records every individual request the engine sent during the run, so users can inspect the exact request and response captured for each call.

Test Reports and Analytics

The system will generate a summary report for each completed test run showing the distribution of HTTP response codes received across all tested endpoints, any server-side failures that were detected, and the proportion of the API's endpoints that were covered during the run. This gives the team a complete picture of how the API performed.

Test Failure Explanation

When a server-side failure or unexpected response is detected, the system will provide a plain-language explanation describing which request triggered the failure, what response was received, and what the likely cause may be. This reduces the time users spend investigating and diagnosing issues after a test run.

Test Execution History

The system will maintain a complete history of all test runs for each project. Users can access the history from the project workspace and see a list of past executions with their timestamps, the test suite used, and a summary of outcomes. Any past run can be opened to review its full results in the same way as a recent run.

Export Test Reports

Users can download the results and report of any completed test run in standard formats such as PDF or CSV. The exported file will contain the full results breakdown, the analytics summary, and the details of any detected failures, allowing users to share test outcomes with stakeholders or include them in project documentation.

2.3.7 Team Collaboration

The system will allow multiple users to collaborate within a shared project. The project owner can invite team members and assign them roles to control what each member can see and do within the project.

Inviting Team Members

The project owner or an admin can invite other users to join the project by entering their email address. The invited user will receive an invitation with a link to accept it. Once accepted, the user will be added to the project and it will appear on their dashboard.

Role Management

Each team member is assigned a role that determines their level of access. An admin can manage project settings, invite or remove members, and execute test runs. A tester can upload specifications, generate test suites, and run tests. A viewer can only see test results and reports without making any changes to the project. The project owner can update a member's role or remove them from the project at any time.

3. System Modeling

3.1 Use Case Diagrams

3.1.1 Level 0 — AutoRestTest Overview

Primary Actors: User (Owner, Admin, Tester, Viewer)

Secondary Actors: LLM service, AutoRestTest engine, Target REST API, Email service

Goal in Context: High-level view of the AutoRestTest platform, an AI-powered SaaS for automated REST API testing. Shows how users manage projects and specifications, generate and run AI-driven test suites, and review results, while the platform coordinates with external services to fulfil these goals.

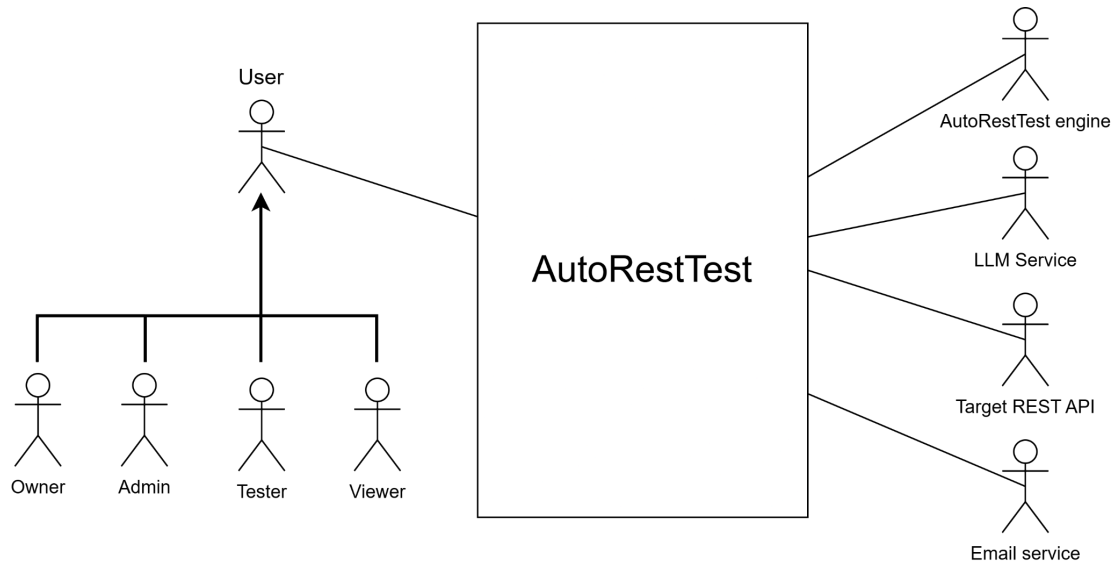


Figure 1: Use case diagram — Level 0: AutoRestTest system overview

3.1.2 Level 1 — Full System

Primary Actors: User (Owner, Admin, Tester, Viewer)

Secondary Actors: LLM service, AutoRestTest engine, Target REST API, Email service

Goal in Context: Expands the overview into all functional areas — authentication, project & endpoint management, specification management, test-suite management, execution, results & reports, and collaboration — showing every major use case and the actors involved.

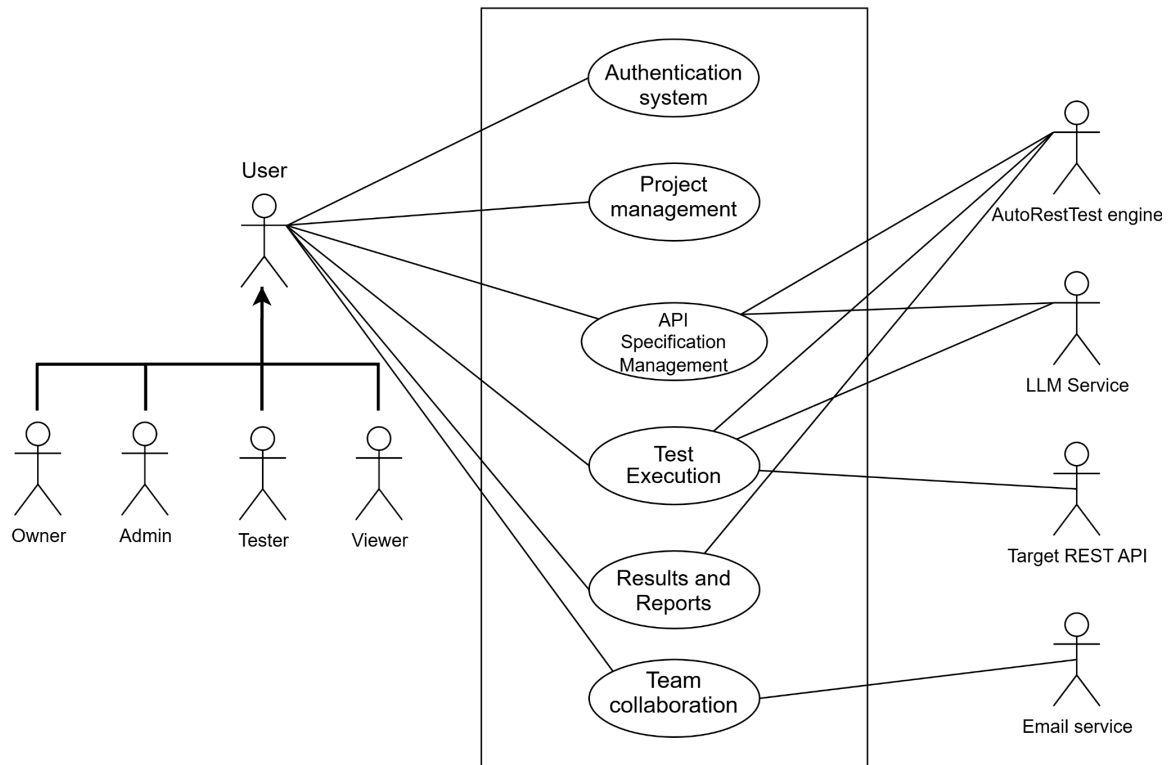


Figure 2: Use case diagram — Level 1: AutoRestTest Full system

3.1.3 Level 1.1 — Authentication system

Primary Actors: user

Secondary Actors: -

Goal in Context: How a user creates an account and authenticates: signing up with a unique email and password, and logging in to obtain a secure session that grants access to the dashboard.

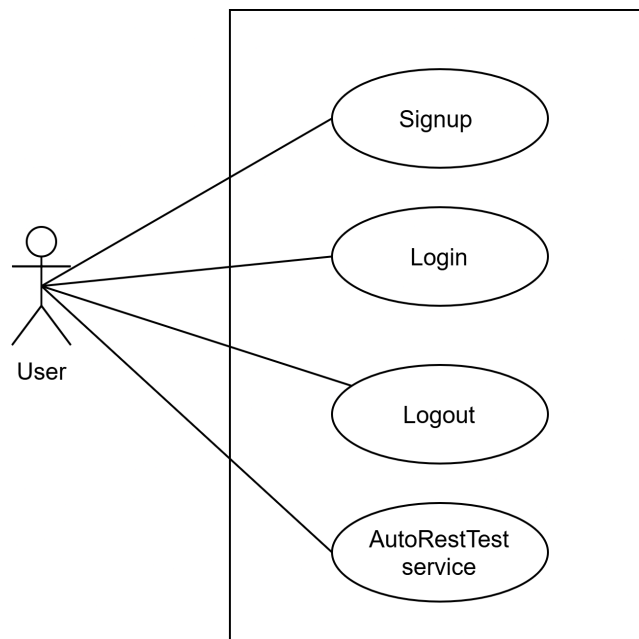


Figure 3: Use case diagram — Level 1.1: Authentication

3.1.4 Level 1.2 — Project Management

Primary Actors: User

Secondary Actors: -

Goal in Context: How an authenticated user creates, opens, renames, and deletes API testing projects from the dashboard, and views the endpoints that belong to a project.

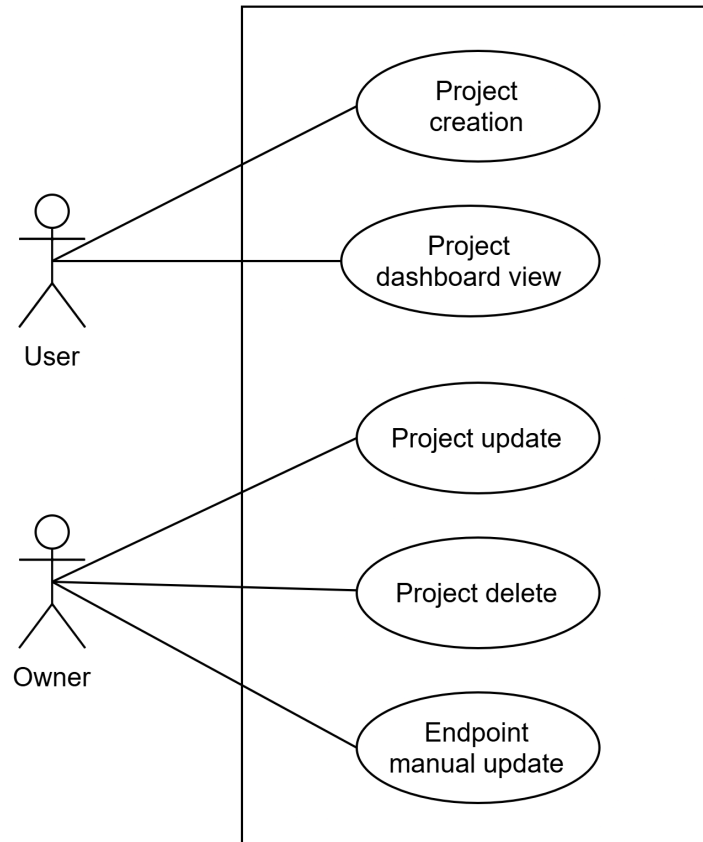


Figure 4: Use case diagram — Level 1.2: Project Management

3.1.5 Level 1.3 — API Specification Management

Primary Actors: User (Admin, Tester, Viewer)

Secondary Actors: LLM service, AutoRestTest engine

Goal in Context: How a user provides the API definition for a project — uploading an OpenAPI 3.0 file that the engine parses to extract endpoints, or generating a specification automatically from source code with AI assistance.

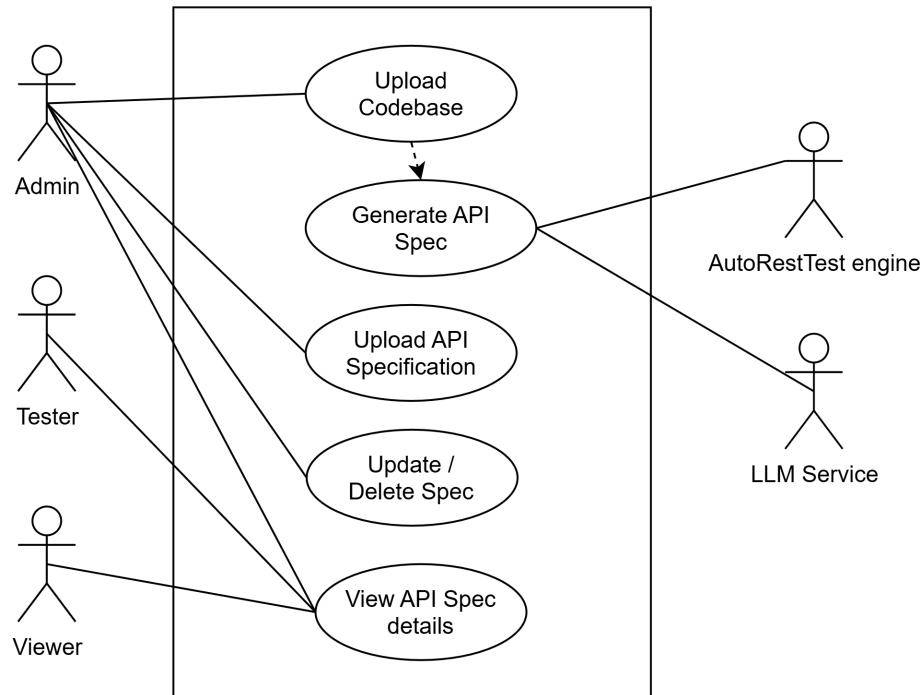


Figure 5: Use case diagram — Level 1.3: API Specification Management

3.1.6 Level 1.4 — Test Suite Management

Primary Actors: User (Admin, Tester, Viewer)

Secondary Actors: LLM service, AutoRestTest engine

Goal in Context: How test suites are produced and managed — automatically generating a comprehensive, dependency-aware suite from the specification, and saving named suites for reuse.

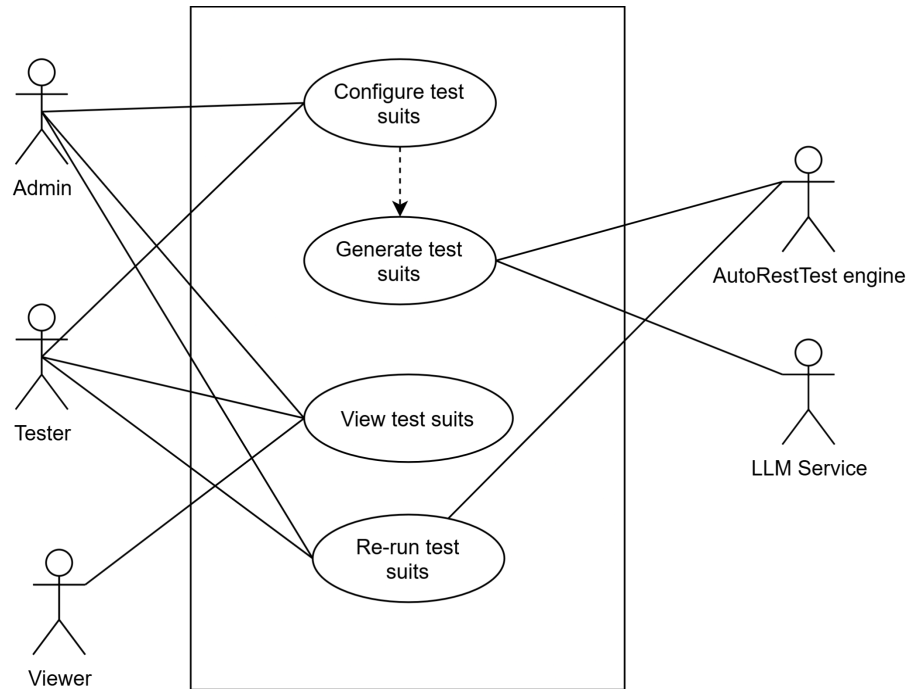


Figure 6: Use case diagram — Level 1.4: Test Suite Management

3.1.7 Level 1.5 — Test Execution

Primary Actors: User (Admin, Tester, Viewer)

Secondary Actors: LLM service, AutoRestTest engine, Target REST API, Email service

Goal in Context: How a user runs a test suite: the platform creates an asynchronous job, the engine sends generated requests to the target API and records responses, and the user is notified when the run completes.

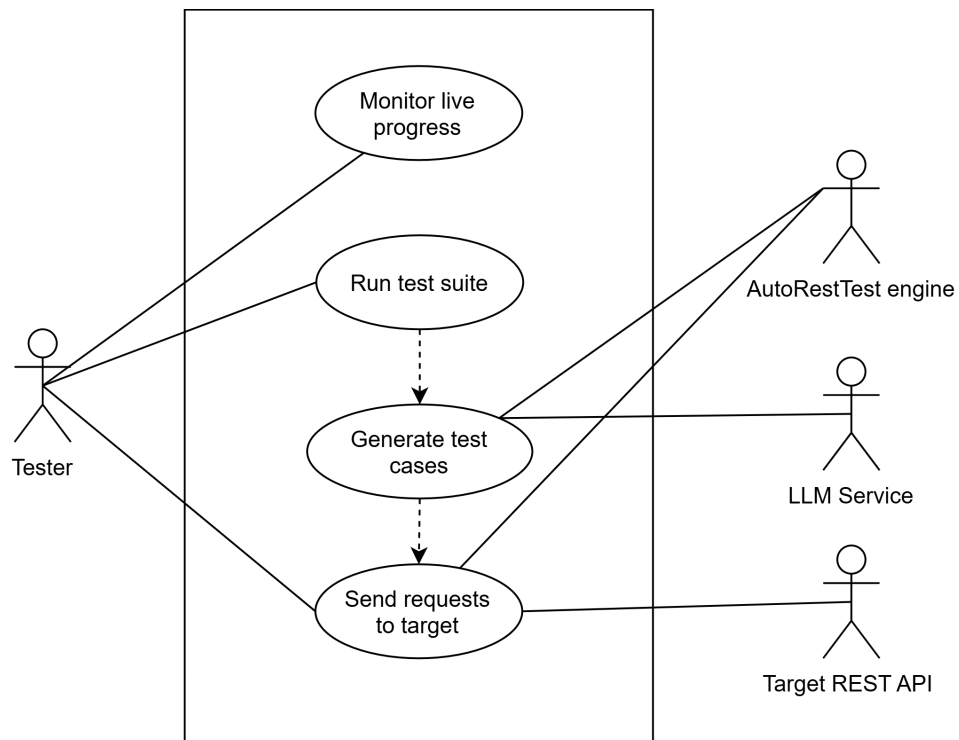


Figure 7: Use case diagram — Level 1.5: Test Execution

3.1.8 Level 1.6 — Test Results & Reports

Primary Actors: User (Admin, Tester, Viewer)

Secondary Actors: LLM service

Goal in Context: How a user reviews a run — per-endpoint results, analytics (status-code distribution, coverage, failures), AI failure explanations, execution history, and exporting reports as PDF/CSV.

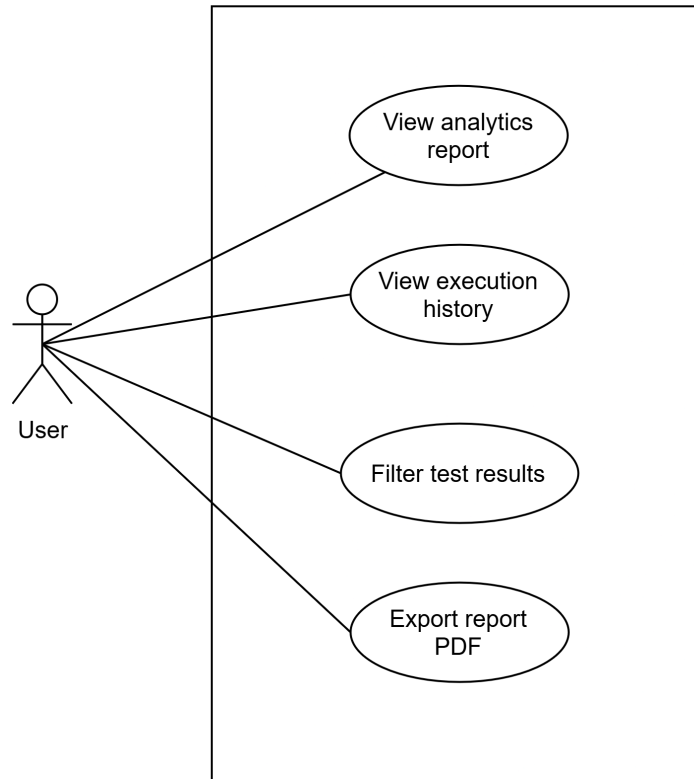


Figure 8: Use case diagram — Level 1.6: Test Results & Reports

3.1.9 Level 1.7 — Team Collaboration

Primary Actors: User (Admin, Tester, Viewer)

Secondary Actors: Email service

Goal in Context: How the project owner invites members by email and assigns roles (admin, tester, viewer), with role-based access controlling what each member can do.

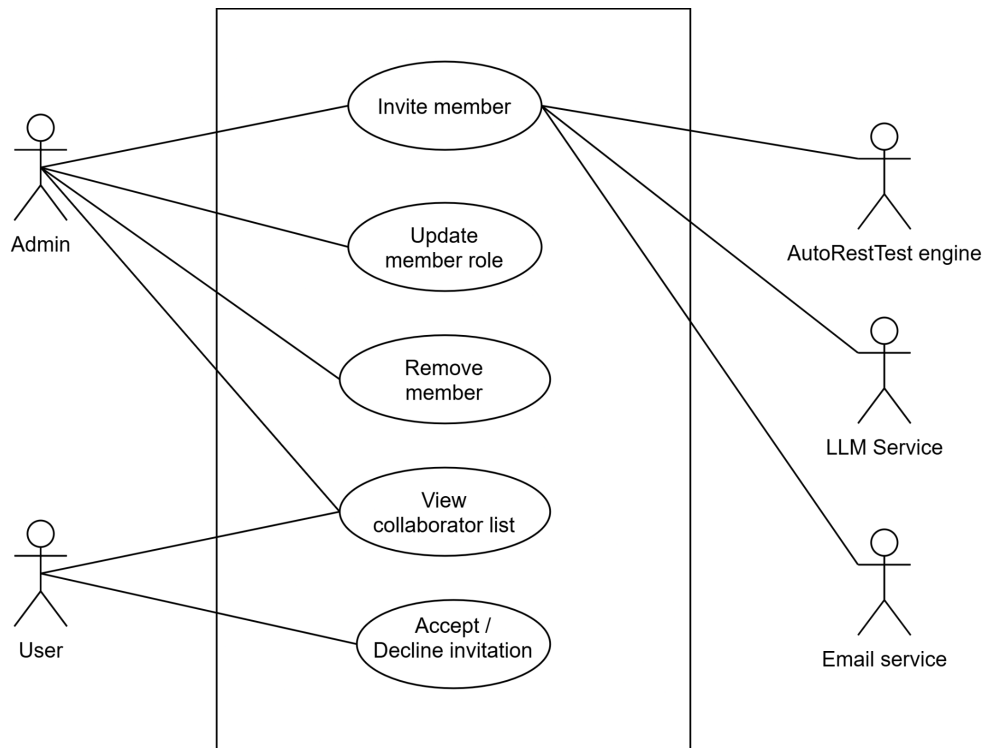


Figure 9: Use case diagram — Level 1.7: Team Collaboration

3.2 Activity Diagrams

3.2.1 Authentication system

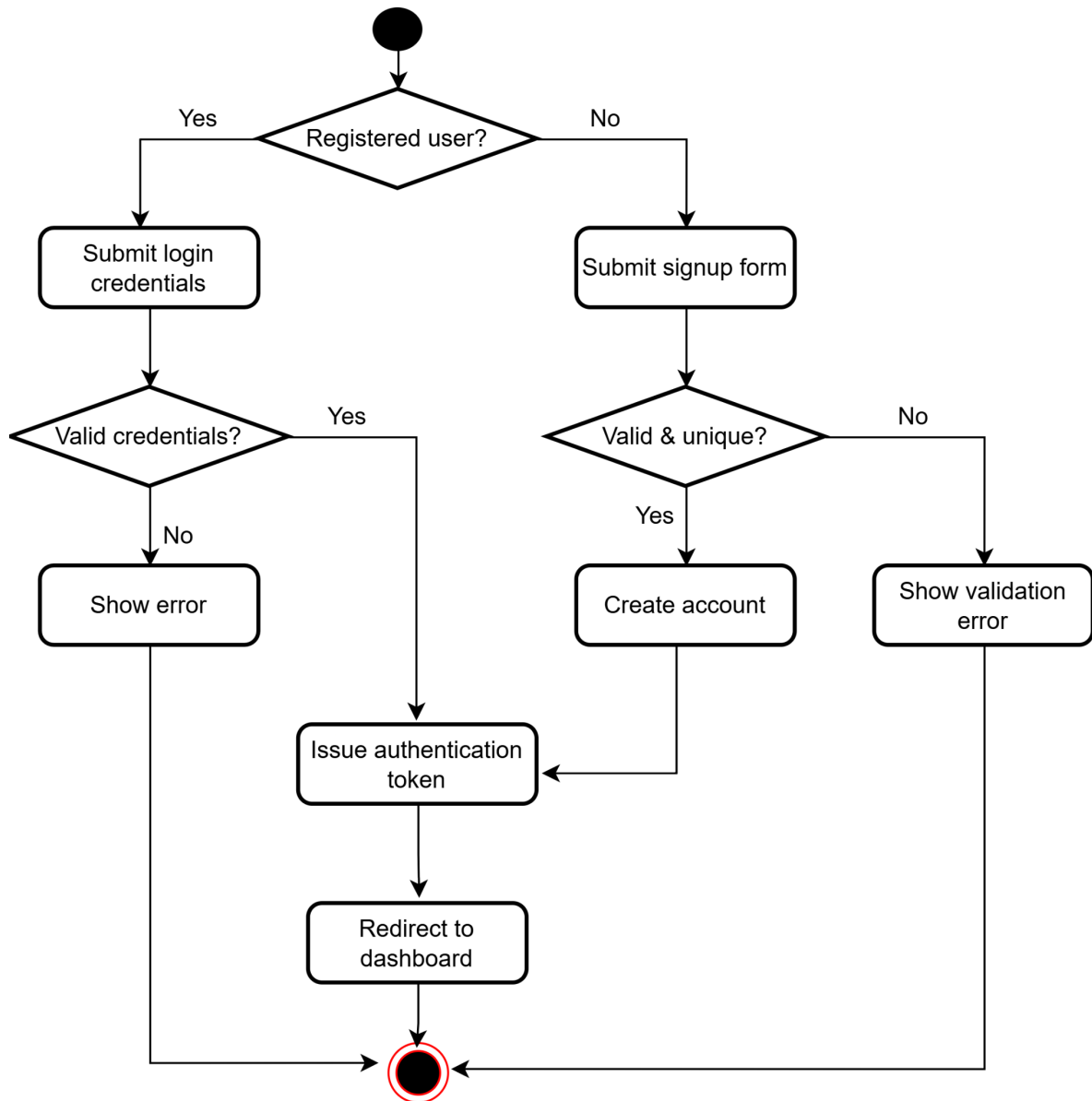


Figure 10: Activity diagram — Authentication

3.2.2 Project Management

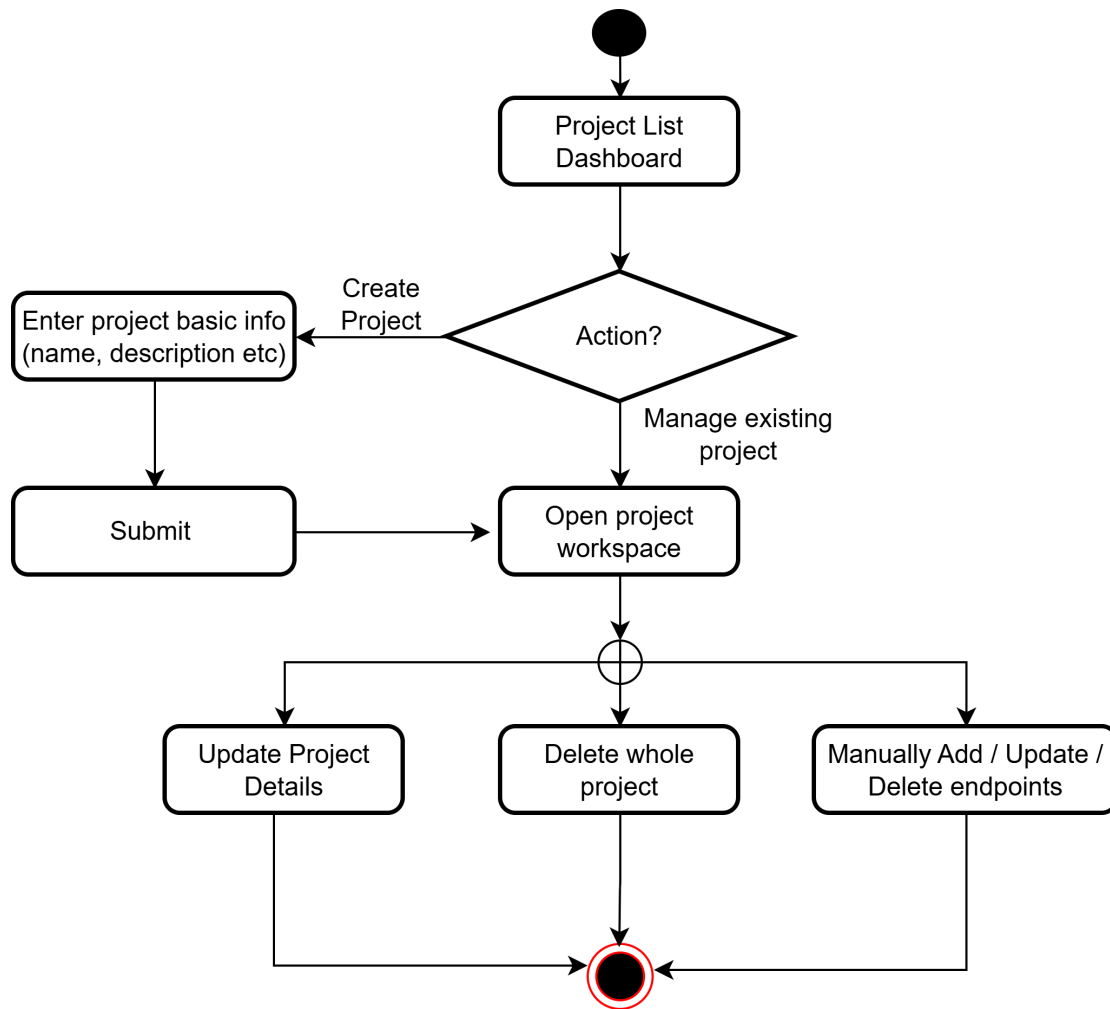


Figure 11: Activity diagram — Project Management

3.2.3 API Specification Management

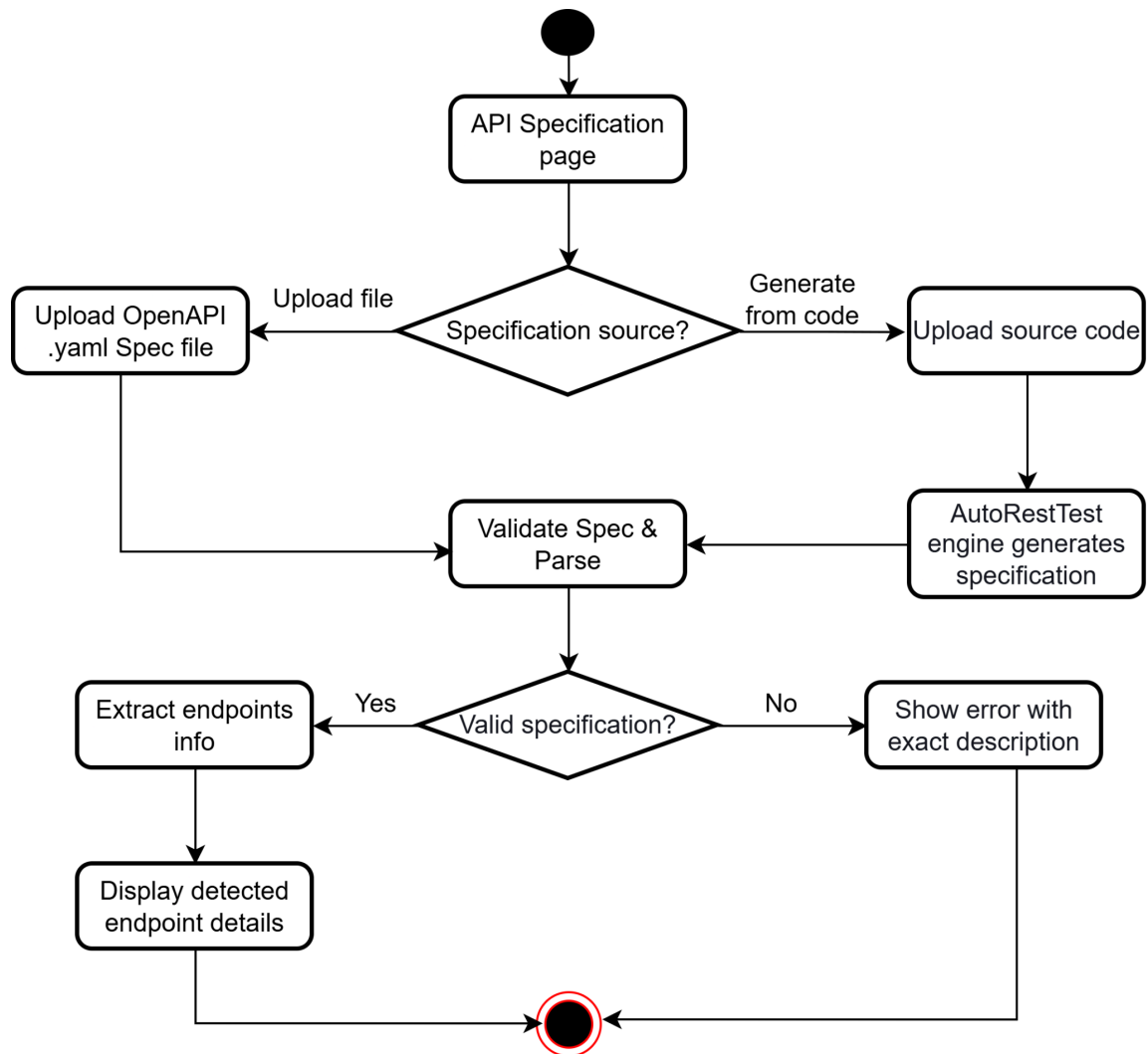


Figure 12: Activity diagram — API Specification Management

3.2.4 Test Suite Management

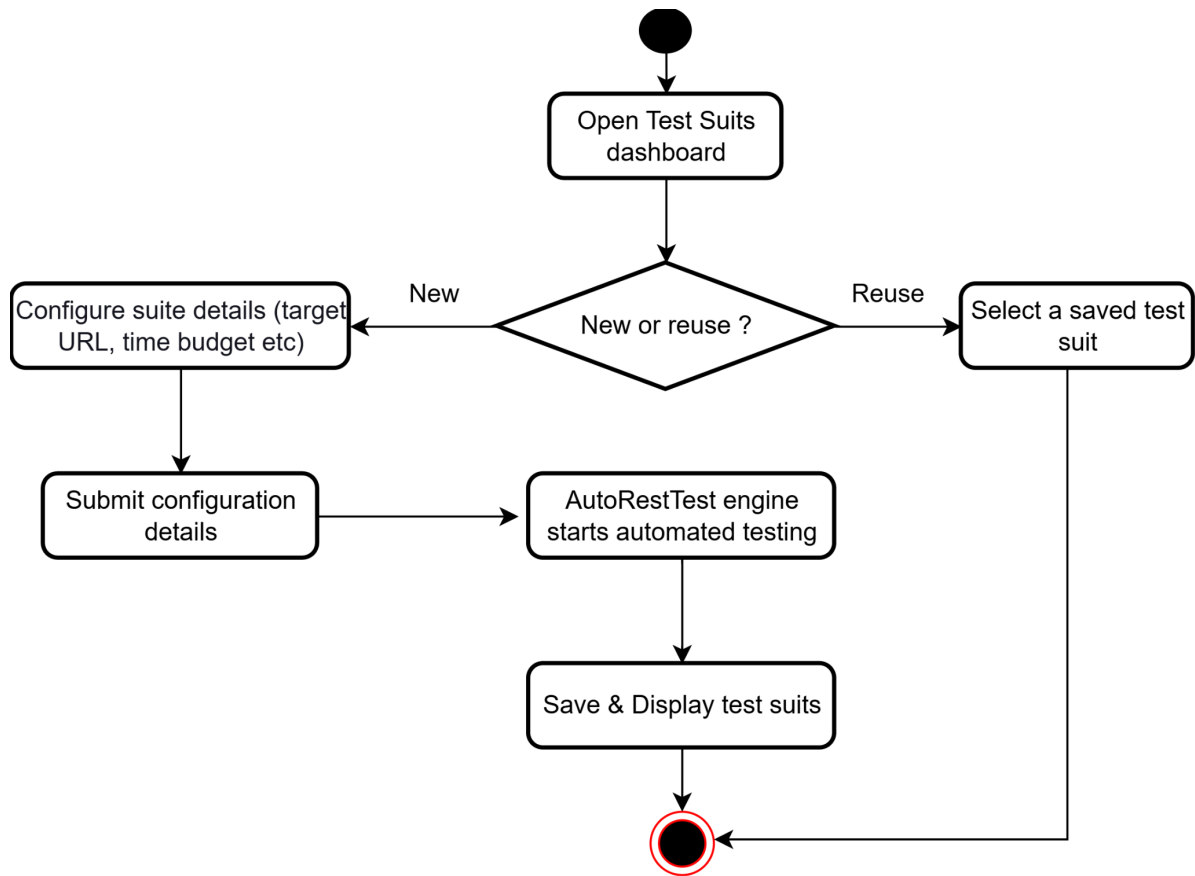


Figure 13: Activity diagram — Test Suite Management

3.2.5 Test Execution

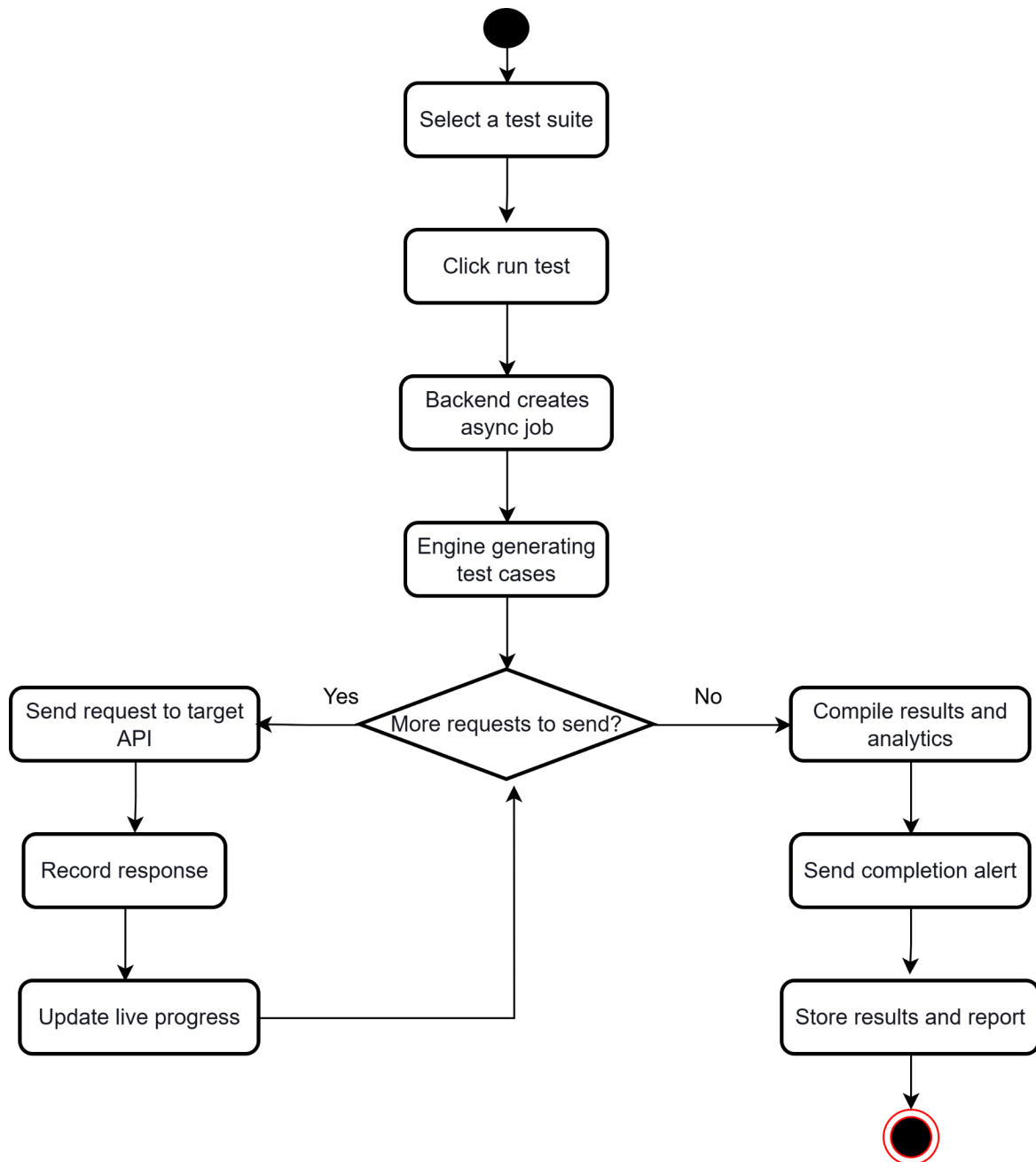


Figure 14: Activity diagram — Test Execution

3.2.6 Test Results & Reports

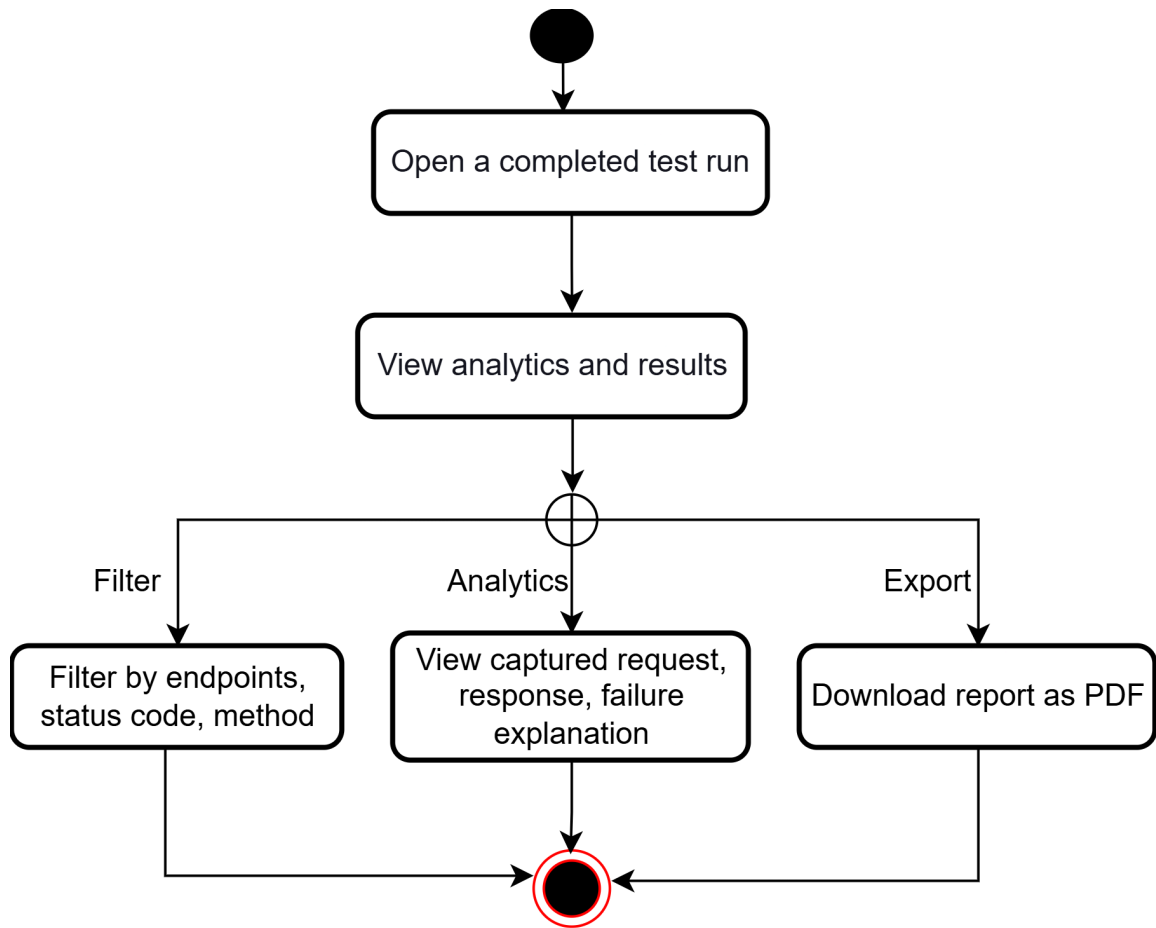


Figure 15: Activity diagram — Test Results & Reports

3.2.7 Team Collaboration

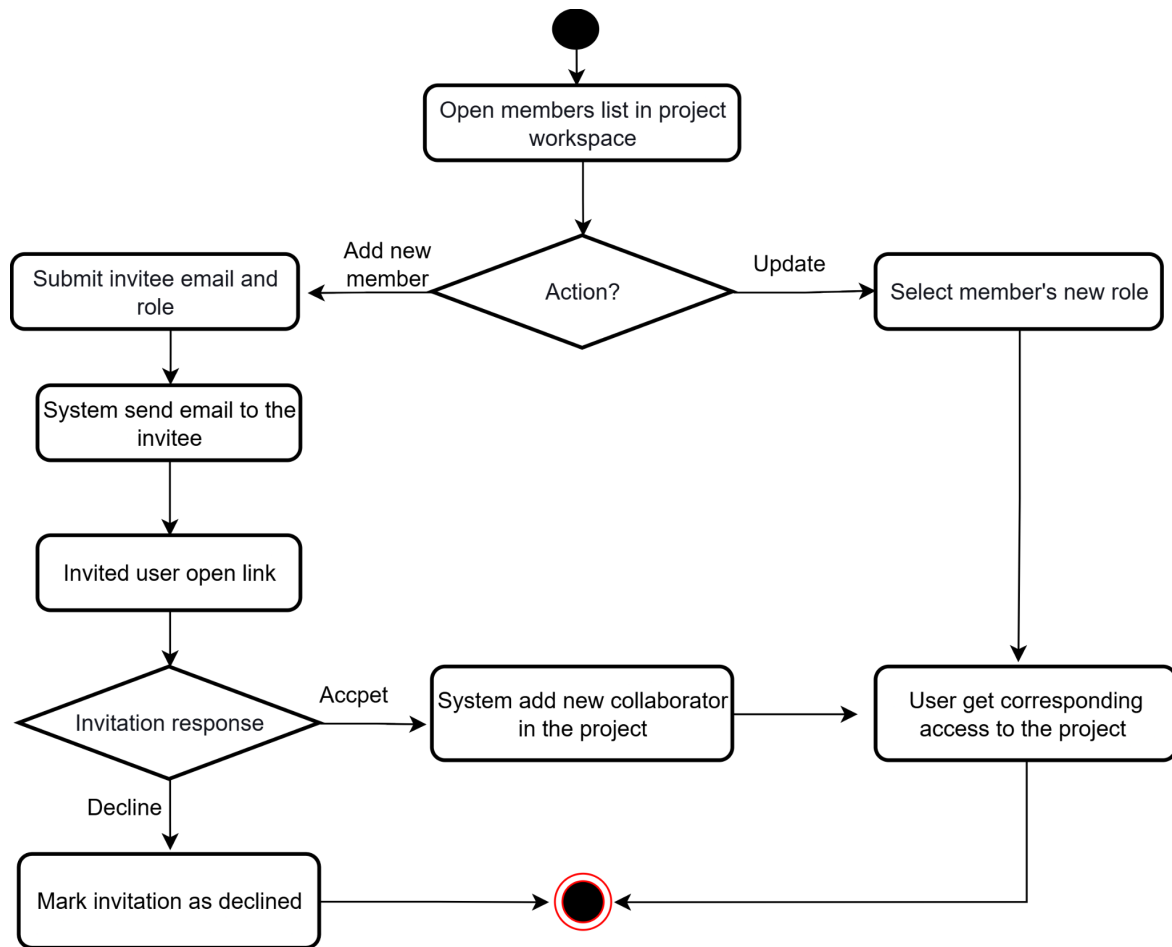


Figure 16: Activity diagram — Team Collaboration

4. Data Modeling

Database modeling describes how the platform's persistent data is structured and related. AutoRestTest uses a relational schema on **PostgreSQL**, accessed through the **Prisma** ORM. The model comprises nine entities and four enumerated types.

4.1 ER Diagram

Entity Relationships:

- A **User** can own many **Projects**.
- A **User** can belong to many **Projects** through **ProjectMember**, a many-to-many association that carries the member's role (each row pairs one user with one project).
- A **Project** can have many **ProjectInvitations**, and each invitation is sent by one **User**.
- A **Project** has at most one **ApiSpecification**.
- A **Project** contains many **Endpoints**.
- A **Project** can have many **TestSuites**, and each **TestSuite** is triggered by one **User**.
- A **TestSuite** produces many **TestCases**, and each **TestCase** tests one **Endpoint**.
- A **TestSuite** captures many **RequestLogs**, and each **RequestLog** maps to at most one **Endpoint**.

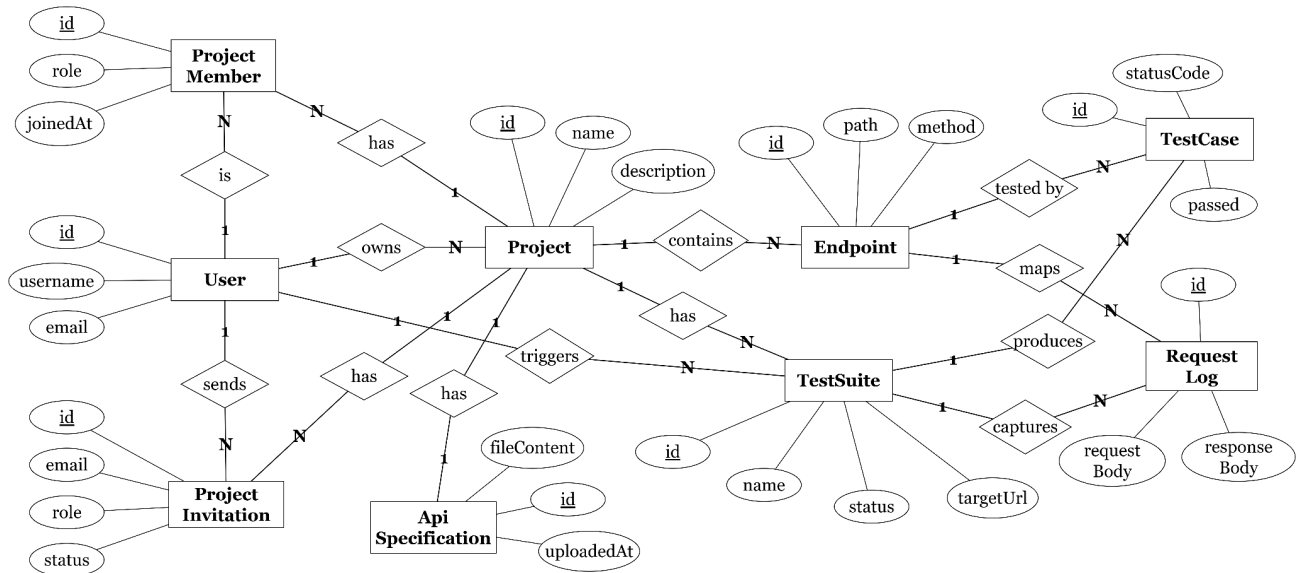


Figure 17: Entity Relationship Diagram

4.2 Database schema

PK = primary key, FK = foreign key, → = references. All identifiers are UUID strings.

Schema	Attribute	Type
User	id (PK)	TEXT (UUID)
	username	TEXT, UNIQUE
	email	TEXT, UNIQUE
	password	TEXT (bcrypt hash)
	createdAt	TIMESTAMP
	updatedAt	TIMESTAMP
Project	id (PK)	TEXT (UUID)
	name	TEXT
	description	TEXT (nullable)
	ownerId (FK)	TEXT → User
	createdAt	TIMESTAMP
	updatedAt	TIMESTAMP
ProjectMember	id (PK)	TEXT (UUID)
	projectId (FK)	TEXT → Project
	userId (FK)	TEXT → User
	role	ENUM(admin, tester, viewer)
	joinedAt	TIMESTAMP
ProjectInvitation	id (PK)	TEXT (UUID)
	projectId (FK)	TEXT → Project
	email	TEXT
	role	ENUM(admin, tester, viewer)
	token	TEXT, UNIQUE
	status	ENUM(pending, accepted, declined, expired)
	invitedById (FK)	TEXT → User
	expiresAt	TIMESTAMP
	createdAt	TIMESTAMP
ApiSpecification	id (PK)	TEXT (UUID)
	projectId (FK)	TEXT → Project, UNIQUE
	fileName	TEXT
	fileContent	TEXT (raw OpenAPI 3.0)
	generatedByAI	BOOLEAN
	uploadedAt	TIMESTAMP
Endpoint	id (PK)	TEXT (UUID)
	projectId (FK)	TEXT → Project
	path	TEXT

Schema	Attribute	Type
	method	ENUM(GET, POST, PUT, PATCH, DELETE)
	description	TEXT (nullable)
	addedManually	BOOLEAN
	createdAt	TIMESTAMP
TestSuite	id (PK)	TEXT (UUID)
	projectId (FK)	TEXT → Project
	name	TEXT (nullable)
	status	ENUM(pending, running, completed, failed)
	jobId	TEXT (nullable)
	targetUrl	TEXT
	timeBudget	INTEGER
	mutationRate	DOUBLE PRECISION
	totalEndpoints	INTEGER
	coveredEndpoints	INTEGER
	totalTestCases	INTEGER
	passedTestCases	INTEGER
	failedTestCases	INTEGER
	triggeredById (FK)	TEXT → User
	startedAt	TIMESTAMP (nullable)
	completedAt	TIMESTAMP (nullable)
	createdAt	TIMESTAMP
TestCase	id (PK)	TEXT (UUID)
	testSuiteId (FK)	TEXT → TestSuite
	endpointId (FK)	TEXT → Endpoint
	requestBody	JSONB (nullable)
	requestHeaders	JSONB (nullable)
	statusCode	INTEGER (nullable)
	responseBody	JSONB (nullable)
	responseTimeMs	INTEGER (nullable)
	passed	BOOLEAN
	failureExplanation	TEXT (nullable)
	createdAt	TIMESTAMP
RequestLog	id (PK)	TEXT (UUID)
	testSuiteId (FK)	TEXT → TestSuite
	endpointId (FK)	TEXT → Endpoint (nullable)

Schema	Attribute	Type
	seq	INTEGER
	method	TEXT
	path	TEXT
	url	TEXT
	statusCode	INTEGER (nullable)
	durationMs	INTEGER (nullable)
	requestHeaders	JSONB (nullable)
	requestBody	TEXT (nullable)
	requestTruncated	BOOLEAN
	responseHeaders	JSONB (nullable)
	responseBody	TEXT (nullable)
	responseTruncated	BOOLEAN
	createdAt	TIMESTAMP

Table – 1: Database schema table

5. AI Engineering Design

AutoRestTest concentrates its intelligence in the testing engine, which transforms a static API description into an adaptive testing process. Rather than treating operation selection, dependency resolution, parameter selection, and value generation as isolated, one-off steps, the engine coordinates them as a single learning system that combines a semantic dependency graph, Large-Language-Model (LLM) value generation, and multi-agent reinforcement learning (MARL).

5.1 AI Pipeline

The AI pipeline turns the target service's OpenAPI description into concrete test requests and refines its own strategy from every server response. As shown in Figure 18, it operates in two phases: an initialization phase that prepares the search space, and an iterative testing-execution phase that explores it within a fixed time budget.

Initialization phase: The pipeline begins by parsing the target service's OpenAPI 3.0 specification, extracting every operation together with its parameters and request/response schemas. From these operations it constructs the Semantic Property Dependency Graph (SPDG) — a directed graph in which each node is an API operation and each edge represents a potential data dependency between two operations. Edges are weighted by the cosine similarity between the names of one operation's output properties and another's input properties, so that operations likely to feed one another (for example, a “create user” call that produces an identifier later consumed by a “get user” call) are linked with a high initial weight. This lets the tool prioritise plausible dependencies instead of blindly matching every input against every output. Finally, the

reinforcement-learning agents are created, each with its own Q-table mapping the choices it can make to their expected reward. During this phase the Value Agent also pre-generates realistic, context-aware values by querying the LLM with few-shot prompts in JSON mode; because this step is not part of the time-bounded loop, it can be performed ahead of testing and cached.

Testing-execution phase: Testing then runs as an iterative loop driven by four cooperating agents, each responsible for a single decision:

Operation Agent — selects the next API operation to exercise.

Parameter Agent — chooses which combination of parameters to include.

Value Agent — generates a value for each parameter, choosing the most effective data source: an LLM-generated value, a value carried over from a dependent operation, or a type-based default.

Dependency Agent — supplies values drawn from previously successful operations, guided by the SPDG.

In the implementation, the Value Agent's responsibilities are further divided across dedicated value, data-source, and request-body agents, with an optional header agent for authenticated targets; the four roles above capture the core decision structure.

Request generation and mutation: On each iteration, the Request Generator assembles a concrete HTTP request from the chosen operation, parameters, values, and headers. A Mutator then perturbs roughly 20% of requests — altering values, types, headers, or methods — specifically to probe error handling and surface faults that valid requests would never reveal. The request is dispatched to the Service Under Test (SUT), and its response is captured and analysed: successful (2xx) responses yield values that are cached for reuse in later requests, while internal server errors (5xx) are recorded for the final report.

The learning loop: The response is the reward signal that closes the loop. Every agent's Q-table is updated using value-decomposition Q-learning with a temporal-difference update, which distributes a shared reward across the agents so that each converges toward its best action while accounting for the others' choices. Action selection follows an ϵ -greedy strategy in which the exploration rate decays over the run — from broad exploration early on to focused exploitation later — balancing the discovery of new behaviours against the reuse of proven ones. In parallel, the SPDG is refined: edge weights are increased for dependencies confirmed by successful responses, decreased for those that fail, and edges that fall below a threshold are pruned, so the dependency model grows more accurate as testing progresses. This cycle repeats until the configured time budget is exhausted, after which the pipeline emits the testing report summarising tested operations, code and operation coverage, the status-code distribution, detected 500-level server errors, and the full execution history.

Because each component addresses a different source of difficulty — the SPDG narrows the dependency search space, the LLM produces domain-realistic inputs, and MARL coordinates the testing strategy over time — they are complementary: the pipeline achieves broader coverage and detects more faults than approaches that optimise any one of these steps in isolation.

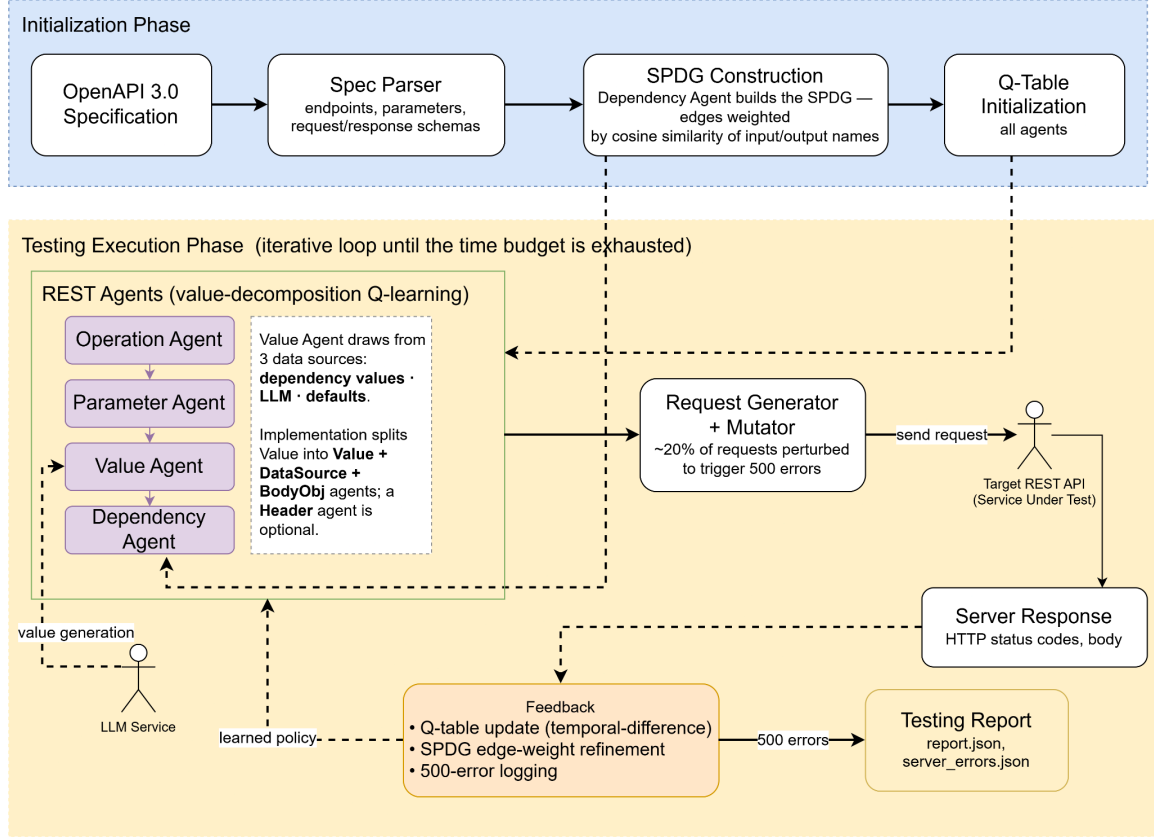


Figure 18: AI system workflow diagram

5.2 Model Selection

AutoRestTest does not depend on any single language model. Because the testing engine communicates with the LLM through a standard OpenAI-compatible REST API, the user is free to choose whichever model best suits their budget and accuracy needs, provided it meets three conditions: it is reachable through an OpenAI-compatible API, it supports structured JSON output (JSON mode), and it is capable of interpreting OpenAPI specifications to generate valid, context-aware request data. Suitable models include OpenAI's GPT family and open-weight models such as gpt-oss-20b (used by default via OpenRouter); other providers such as Google Gemini can also be used when accessed through an OpenAI-compatibility endpoint. The selected model is set in a single configuration entry, so switching models — for example, moving from a free open-weight model during development to a stronger frontier model for higher-fidelity

value generation — requires only a configuration change and no modification to the engine.

5.3 Prompt Design

The engine relies on carefully engineered prompts to drive its LLM interactions. Rather than one general prompt, it uses a family of task-specific prompts that share a few common design principles: each injects the operation summary together with its OpenAPI schema, instructs the model to return a strict JSON structure (paired with the model's JSON mode), and supplies few-shot examples so the output matches the expected format. These prompts fall into four groups:

- Value-generation prompts: produce valid, context-aware values for an operation's parameters and request body.
- Feedback / retry prompts: regenerate values after a failed request by conditioning on the server's response, dropping inputs that caused the failure.
- Classification prompts: label failures (for example, parameter-constraint, format, or dependency errors) so the engine can respond appropriately.
- Utility prompts: handle supporting tasks such as repairing malformed JSON and identifying authentication parameters.

Here is some example:

```
CLASSIFICATION_PROMPT = "" Given an error message, classify the error as one of
"PARAMETER CONSTRAINT, FORMAT, PARAMETER DEPENDENCY,
OPERATION DEPENDENCY". ""
```

```
REQUEST_BODY_GEN_PROMPT = """" Given a summary of an operation its request
body schema from its OpenAPI Specification, generate a valid context-aware request
body for the operation. Return the answer as a JSON object with the following structure:
{ "request_body": [correct request body] } In the case where the request body is an
object, return a correctly formatted object as the [correct request body] value. In the
object, include all required fields as specified from the OpenAPI Specification for object
request bodies, but otherwise include/exclude optional properties to ensure the object is
accepted. In the case where the request body is an array, use a list as the request_body
value. Do not solely rely on the given constraint values, and ensure you read the
associated descriptions for maximum accuracy. Use any provided example values to
guide your generation formatting."""
```

```
IDENTIFY_AUTHENTICATION_SYSTEM_MESSAGE = "You will receive a
summary of an operation and its full schema from the OpenAPI Specification. Your goal
is to determine if the operation consists of any authentication information sent as
parameters in either the query or the request body. When referring to request body
parameters, identify object properties that might be used for authentication."
```

Authentication information specifically refers to either usernames or passwords (or similar). Return your result in the specified JSON format."

6. Preliminary Test Plan

6.1 High-Level Description of Testing Goals

The testing goals for this platform are represented below from a high level:

- To verify that AutoRestTest meets its most important functional and non-functional requirements.
- To ensure that users can securely register, authenticate, and access only the projects they are authorized for.
- To confirm that uploaded OpenAPI specifications are parsed correctly and that endpoints are extracted accurately.
- To ensure that the AI testing engine generates valid requests and executes test suites reliably.
- To verify that test results, analytics, and LLM-based failure explanations are accurate and clearly presented.
- To confirm that reports can be exported (PDF/CSV) and that test-execution history is preserved.
- To guarantee that team collaboration and role-based access control work correctly and securely.

6.2 Test Cases

The test cases for the testing of this platform are documented below:

Test Case ID	Title	Test Scenario	Expected Outcome
T1	User Registration and Login	- Register a new account with a valid name, email, and password. - Log in using the same credentials, then retry with a wrong password.	The account is created with a securely hashed password, a JWT token is issued on valid login and the user reaches the dashboard, while invalid credentials are rejected.
T2	Project Creation and Access Control	- Create a new project as its owner.	The project is created with the creator as admin and appears on the owner's dashboard, while the

		- Attempt to open the same project from an unrelated account.	unauthorized account is denied access.
T3	OpenAPI Specification Upload and Parsing	- Upload a valid OpenAPI 3.0 specification to a project. - Then upload a malformed specification file.	The valid specification is stored and its endpoints (path and method) are extracted and listed correctly, while the malformed file is rejected with a clear error message.
T4	Automatic Test Suite Generation	- Trigger automatic test-suite generation for a parsed specification.	A test suite containing test cases that cover the specification's operations is generated and saved so it can be reused later.
T5	One-Click Test Execution	- Run a generated test suite with a single action. - Observe the suite status while it runs.	The backend creates an asynchronous job, the suite status transitions from pending to running to completed, and the interface stays responsive throughout.
T6	Viewing and Filtering Test Results	- Open a completed test run. - Filter the results by endpoint, status, and response code.	The results (endpoints, methods, responses, and pass/fail outcomes) are displayed, and each filter returns the correct subset of test cases.
T7	Captured Request and Response Inspection	- Select a test case from a completed run and inspect its captured request and response.	The full request (method, URL, headers, and body) and the response (status code and body) are shown exactly as they were logged.

T8	Analytics and LLM Failure Explanation	- View the analytics for a run (response-code distribution, coverage, and server failures). - Request an explanation for a failed test case.	The analytics accurately reflect the run and the LLM produces a relevant, understandable explanation for the failure.
T9	Exporting Test Reports	- Export a completed run's report as PDF and as CSV.	Both files download successfully and contain the run's results accurately.
T10	Team Collaboration and Role Management	- Invite a user by email with an assigned role (tester or viewer). - Accept the invitation and attempt an action beyond the assigned role.	An invitation with a unique token is created, the invited user becomes a member with the assigned role on acceptance, and actions outside that role are restricted.

Table – 2: Preliminary Test Plan

7. Timeline

The estimated timeline for the AutoRestTest project is presented below:

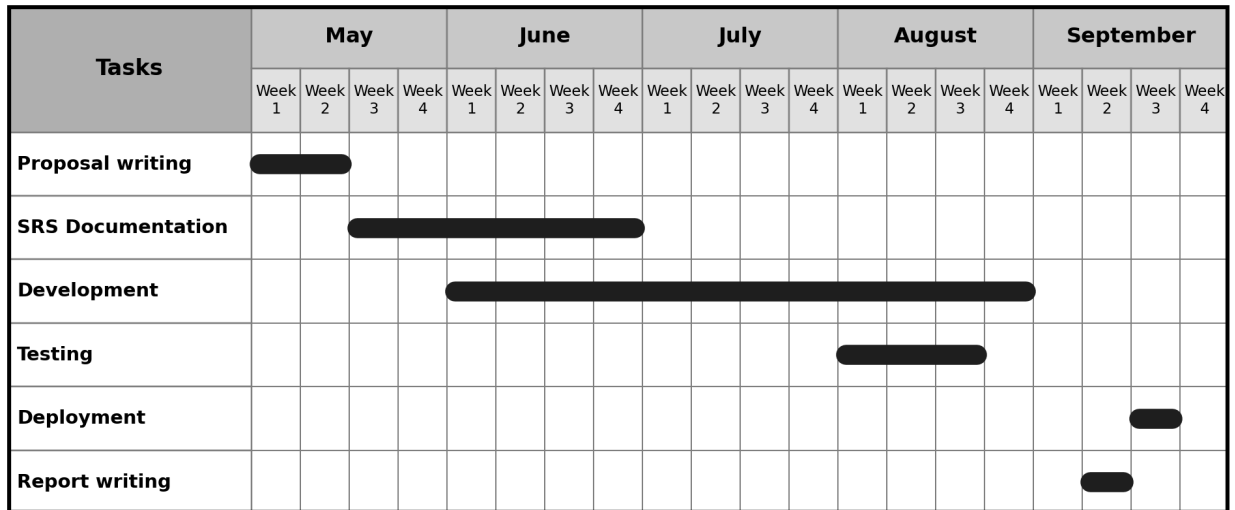


Figure 19: Timeline Gantt Chart

8. Conclusion

This report has presented the complete technical specification of AutoRestTest, an AI-powered collaborative platform for automated REST API testing. It covered the

project's problem statement, objectives, scope, and deliverables; a requirements analysis through QFD, stakeholders, and usage scenarios; system and data modeling; and a preliminary test plan. Diagrams and tables have been used throughout to make the design clear. While the platform already supports the full testing workflow, it can be extended in future with support for other API paradigms (GraphQL, SOAP, gRPC), load and performance testing, and more advanced RL/LLM techniques. This document should provide a complete understanding of the requirements and technical design of AutoRestTest.

9. References

[1] Tyler Stennett, Myeongsoo Kim, Saurabh Sinha, and Alessandro Orso, A Multi-Agent Approach for REST API Testing with Semantic Graphs and LLM-Driven Inputs, in Proceedings of 47th International Conference on Software Engineering (ICSE 2025)

Paper: <https://arxiv.org/abs/2411.07098v2>